TASK 10: ADVANCED SQL QUERY SYSTEM

STUDENT DETAILS

Name: Divya. V
Roll No: E25AI111
Department: B.Sc Computer Science With AI

SCENARIO

The Artisan Crumb Bakery & Pastry Management System wants to perform advanced analysis on customer purchases, bakery item pricing, and sales performance. The management team needs complex reports to identify high-value customers, premium bakery items, and business trends.

Students will implement **Subqueries and Nested Queries** to extract deeper insights from the Bakery Database.

1. Implement Subqueries and Nested Queries

Concept

A Subquery is a query written inside another SQL query. It helps solve complex problems by using the result of one query in another query.

Types Covered

- Single-row Subquery
- Multi-row Subquery
- Nested Subquery

Example

Step 1: Find average bakery item price

```
SQL> SELECT AVG(Price)
  2  FROM Product;

AVG(PRICE)
----------
591.666667
```

Step 2: Compare bakery items with average price

```
SQL> SELECT Title, Price
  2  FROM Product
  3  WHERE Price >
  4  (
  5      SELECT AVG(Price)
  6      FROM Product
  7  );

TITLE                          PRICE
------------------------- --------
Special Wedding Cake       1200.00
Chocolate Fudge Cake        750.00
Black Forest Cake           600.00
```

2. Find Products Above Average Price

The bakery wants to identify premium bakery items. Students should write queries

1. Find bakery items priced above the average price.

```
SQL> SELECT Product_ID,
  2         Title,
  3         Price
  4  FROM Product
  5  WHERE Price >
  6  (
  7      SELECT AVG(Price)
  8      FROM Product
  9  );

PRODUCT_ID TITLE                          PRICE
---------- ------------------------- --------
P006       Special Wedding Cake       1200.00
P001       Chocolate Fudge Cake        750.00
P005       Black Forest Cake           600.00
```

2. Display bakery item name, category, and price.

```
SQL> SELECT P.Title,
  2         C.Category_Name,
  3         P.Price
  4  FROM Product P
  5  JOIN Category C
  6  ON P.Category_ID = C.Category_ID
  7  WHERE P.Price >
  8  (
  9      SELECT AVG(Price)
 10      FROM Product
 11  );

TITLE                     CATEGORY_NAME          PRICE
------------------------- -------------------- --------
Black Forest Cake         Celebration Cakes      600.00
Chocolate Fudge Cake      Celebration Cakes      750.00
Special Wedding Cake      Celebration Cakes     1200.00
```

3. Identify expensive bakery items in each category.

```
SQL> SELECT P.Product_ID,
  2         P.Title,
  3         C.Category_Name,
  4         P.Price
  5  FROM Product P
  6  JOIN Category C
  7  ON P.Category_ID = C.Category_ID
  8  WHERE P.Price >
  9  (
 10      SELECT AVG(P2.Price)
 11      FROM Product P2
 12      WHERE P2.Category_ID = P.Category_ID
 13  );

PRODUCT_ID TITLE                     CATEGORY_NAME          PRICE
---------- ------------------------- -------------------- --------
P006       Special Wedding Cake      Celebration Cakes     1200.00
```

4. Find bakery items with price higher than a selected category average.

```
SQL> SELECT P.Product_ID,
  2  P.Title,
  3  P.Price,
  4  C.Category_Name
  5  FROM Product P
  6  JOIN Category C
  7  ON P.Category_ID = C.Category_ID
  8  WHERE P.Price >
  9  (
 10  SELECT AVG(P2.Price)
 11  FROM Product P2
 12  WHERE P2.Category_ID = P.Category_ID
 13  );

PRODUCT_ID TITLE                       PRICE CATEGORY_NAME
---------- ------------------------ -------- --------------------
P006       Special Wedding Cake       1200.00 Celebration Cakes
```

3. Identify Customers with Maximum Purchases

Analyze customer purchasing behavior.

1. Customer who spent the highest amount.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         SUM(O.Total_Amount) AS Total_Spending
  4  FROM Customer C
  5  JOIN Orders O
  6    ON C.Customer_ID = O.Customer_ID
  7  GROUP BY C.Customer_ID,
  8           C.First_Name,
  9           C.Last_Name
 10  HAVING SUM(O.Total_Amount) =
 11  (
 12      SELECT MAX(Total_Sales)
 13      FROM
 14      (
 15          SELECT SUM(Total_Amount) AS Total_Sales
 16          FROM Orders
 17          GROUP BY Customer_ID
 18      )
 19  );

CUSTOMER_ID          CUSTOMER_NAME                                TOTAL_SPENDING
-------------------- ---------------------------------------- --------------
C001                 John Doe                                           3590
```

2. Customers with maximum number of orders.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         COUNT(O.Order_ID) AS Number_Of_Orders
  4  FROM Customer C
  5  JOIN Orders O
  6    ON C.Customer_ID = O.Customer_ID
  7  GROUP BY C.Customer_ID,
  8           C.First_Name,
  9           C.Last_Name
 10  HAVING COUNT(O.Order_ID) =
 11  (
 12      SELECT MAX(Order_Count)
 13      FROM
 14      (
 15          SELECT COUNT(*) AS Order_Count
 16          FROM Orders
 17          GROUP BY Customer_ID
 18      )
 19  );

CUSTOMER_ID          CUSTOMER_NAME                                NUMBER_OF_ORDERS
-------------------- ---------------------------------------- ----------------
C001                 John Doe                                                3
```

3. Customers whose spending is above average.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         SUM(O.Total_Amount) AS Total_Spending
  4  FROM Customer C
  5  JOIN Orders O
  6    ON C.Customer_ID = O.Customer_ID
  7  GROUP BY C.Customer_ID,
  8           C.First_Name,
  9           C.Last_Name
 10  HAVING SUM(O.Total_Amount) >
 11  (
 12      SELECT AVG(Customer_Spending)
 13      FROM
 14      (
 15          SELECT SUM(Total_Amount) AS Customer_Spending
 16          FROM Orders
 17          GROUP BY Customer_ID
 18      )
 19  );

CUSTOMER_ID          CUSTOMER_NAME                            TOTAL_SPENDING
-------------------- ---------------------------------------- --------------
C001                 John Doe                                           3590
C003                 Alice Brown                                        1450
```

4. Top 5 valuable customers.

```
SQL> SELECT *
  2  FROM
  3  (
  4      SELECT C.Customer_ID,
  5             C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  6             SUM(O.Total_Amount) AS Total_Spending
  7      FROM Customer C
  8      JOIN Orders O
  9        ON C.Customer_ID = O.Customer_ID
 10      GROUP BY C.Customer_ID,
 11               C.First_Name,
 12               C.Last_Name
 13      ORDER BY Total_Spending DESC
 14  )
 15  WHERE ROWNUM <= 5;

CUSTOMER_ID          CUSTOMER_NAME                          TOTAL_SPENDING
-------------------- -------------------------------------- --------------
C001                 John Doe                                         3590
C003                 Alice Brown                                      1450
C004                 Bob White                                        1250
C005                 Charlie Green                                     950
C002                 Jane Smith                                        720
```

4. Generate Complex Business Queries

Query 1: Best-Selling Bakery Item

Display bakery item name, total quantity sold and total revenue generated.

```
SQL> SELECT P.Title AS Bakery_Item_Name,
  2         SUM(OD.Quantity) AS Total_Quantity_Sold,
  3         SUM(OD.Quantity * OD.Price) AS Total_Revenue
  4  FROM Product P
  5  JOIN Order_Details OD
  6    ON P.Product_ID = OD.Product_ID
  7  GROUP BY P.Product_ID,
  8           P.Title
  9  HAVING SUM(OD.Quantity) =
 10  (
 11      SELECT MAX(Qty_Sold)
 12      FROM
 13      (
 14          SELECT SUM(Quantity) AS Qty_Sold
 15          FROM Order_Details
 16          GROUP BY Product_ID
 17      )
 18  );

BAKERY_ITEM_NAME          TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------------- ------------------- -------------
Chocolate Fudge Cake                        3       2250.00
Red Velvet Pastry                           3       1350.00
```

Query 2: High-Value Customers

Find customers whose total purchase amount is greater than average customer spending.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         SUM(O.Total_Amount) AS Total_Spending
  4  FROM Customer C
  5  JOIN Orders O
  6    ON C.Customer_ID = O.Customer_ID
  7  GROUP BY C.Customer_ID,
  8           C.First_Name,
  9           C.Last_Name
 10  HAVING SUM(O.Total_Amount) >
 11  (
 12      SELECT AVG(Customer_Total)
 13      FROM
 14      (
 15          SELECT SUM(Total_Amount) AS Customer_Total
 16          FROM Orders
 17          GROUP BY Customer_ID
 18      )
 19  );

CUSTOMER_ID          CUSTOMER_NAME                              TOTAL_SPENDING
-------------------- ---------------------------------------- --------------
C001                 John Doe                                         3590
C003                 Alice Brown                                      1450
```

Query 3: Category Performance Analysis

Highest revenue-generating category.

```
SQL> SELECT C.Category_Name,
  2         SUM(OD.Quantity * OD.Price) AS Total_Revenue
  3  FROM Category C
  4  JOIN Product P
  5    ON C.Category_ID = P.Category_ID
  6  JOIN Order_Details OD
  7    ON P.Product_ID = OD.Product_ID
  8  GROUP BY C.Category_ID,
  9           C.Category_Name
 10  HAVING SUM(OD.Quantity * OD.Price) =
 11  (
 12      SELECT MAX(Category_Revenue)
 13      FROM
 14      (
 15          SELECT SUM(OD2.Quantity * OD2.Price) AS Category_Revenue
 16          FROM Category C2
 17          JOIN Product P2
 18            ON C2.Category_ID = P2.Category_ID
 19          JOIN Order_Details OD2
 20            ON P2.Product_ID = OD2.Product_ID
 21          GROUP BY C2.Category_ID
 22      )
 23  );

CATEGORY_NAME        TOTAL_REVENUE
-------------------- -------------
Celebration Cakes          3450.00
```

Number of products sold in each category.

```
SQL> SELECT C.Category_Name,
  2         SUM(OD.Quantity) AS Products_Sold
  3  FROM Category C
  4  JOIN Product P
  5    ON C.Category_ID = P.Category_ID
  6  JOIN Order_Details OD
  7    ON P.Product_ID = OD.Product_ID
  8  GROUP BY C.Category_ID,
  9           C.Category_Name
 10  ORDER BY Products_Sold DESC;

CATEGORY_NAME        PRODUCTS_SOLD
-------------------- -------------
Celebration Cakes                5
Artisan Breads                   3
Morning Pastries                 2
```

Average bakery item price by category.

```
SQL> SELECT C.Category_Name,
  2         ROUND(AVG(P.Price), 2) AS Average_Price
  3  FROM Category C
  4  JOIN Product P
  5    ON C.Category_ID = P.Category_ID
  6  GROUP BY C.Category_ID,
  7           C.Category_Name
  8  ORDER BY Average_Price DESC;

CATEGORY_NAME        AVERAGE_PRICE
-------------------- -------------
Celebration Cakes              850
Artisan Breads                 450
Morning Pastries               350
```

Query 4: Customer Purchase History

Display Customer Name, Number of Orders, Total Purchase Amount and Most Purchased Bakery Item.

```
SQL> SELECT C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  2         COUNT(DISTINCT O.Order_ID) AS Number_Of_Orders,
  3         SUM(O.Total_Amount) AS Total_Purchase_Amount,
  4         MAX(TP.Title) AS Most_Purchased_Product
  5  FROM Customer C
  6  JOIN Orders O ON C.Customer_ID = O.Customer_ID
  7  LEFT JOIN (
  8      SELECT O2.Customer_ID,
  9             P.Title,
 10             ROW_NUMBER() OVER (PARTITION BY O2.Customer_ID ORDER BY SUM(OD.Quantity) DESC) as rn
 11      FROM Orders O2
 12      JOIN Order_Details OD ON O2.Order_ID = OD.Order_ID
 13      JOIN Product P ON OD.Product_ID = P.Product_ID
 14      GROUP BY O2.Customer_ID, P.Title
 15  ) TP ON C.Customer_ID = TP.Customer_ID AND TP.rn = 1
 16  GROUP BY C.Customer_ID,
 17           C.First_Name,
 18           C.Last_Name;

CUSTOMER_NAME        NUMBER_OF_ORDERS TOTAL_PURCHASE_AMOUNT MOST_PURCHASED_PRODUCT
-------------------- ---------------- -------------------- ------------------------
Jane Smith                          1               720.00 Sourdough Bread
John Doe                            3              3590.00 Chocolate Fudge Cake
Bob White                           1              1250.00 Black Forest Cake
Charlie Green                       1               950.00
Alice Brown                         1              1450.00 Red Velvet Pastry
David Miller                        1               500.00

6 rows selected.
```

5. Prepare Advanced SQL Report

Report 1: Premium Product Report

Display

- Product Name
- Category
- Price
- Stock Availability

Condition: Products above average price

```
SQL> SELECT P.Title,
  2         C.Category_Name,
  3         P.Price,
  4         I.Stock_Status
  5  FROM Product P
  6  JOIN Category C ON P.Category_ID = C.Category_ID
  7  JOIN Inventory I ON P.Product_ID = I.Product_ID
  8  WHERE P.Price >
  9  (
 10      SELECT AVG(Price)
 11      FROM Product
 12  );

TITLE                     CATEGORY_NAME           PRICE STOCK_STATUS
------------------------- ------------------- -------- --------------
Black Forest Cake         Celebration Cakes      600.00 Out of Stock
Chocolate Fudge Cake      Celebration Cakes      750.00 In Stock
Special Wedding Cake      Celebration Cakes     1200.00 In Stock
```

Report 2: Customer Value Report

Display

- Customer Name
- Total Orders
- Total Spending
- Customer Category

```
SQL> SELECT C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  2         COUNT(O.Order_ID) AS Total_Orders,
  3         SUM(O.Total_Amount) AS Total_Spending,
  4         CASE
  5             WHEN SUM(O.Total_Amount) >= 10000 THEN 'Premium'
  6             WHEN SUM(O.Total_Amount) >= 5000 THEN 'Gold'
  7             ELSE 'Regular'
  8         END AS Customer_Category
  9  FROM Customer C
 10  JOIN Orders O ON C.Customer_ID = O.Customer_ID
 11  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
 12  ORDER BY Total_Spending DESC;

CUSTOMER_NAME        TOTAL_ORDERS TOTAL_SPENDING CUSTOME
-------------------- ------------ -------------- -------
John Doe                        3           3590 Regular
Alice Brown                     1           1450 Regular
Bob White                       1           1250 Regular
Charlie Green                   1            950 Regular
Jane Smith                      1            720 Regular
David Miller                    1            500 Regular

6 rows selected.
```

Report 3: Sales Performance Report

Product-wise sales

```
SQL> SELECT P.Title,
  2         SUM(OD.Quantity) AS Quantity_Sold
  3  FROM Product P
  4  JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  5  GROUP BY P.Product_ID, P.Title;

TITLE                     QUANTITY_SOLD
------------------------- -------------
Chocolate Fudge Cake                  3
Sourdough Bread                       2
Red Velvet Pastry                     3
Black Forest Cake                     2
```

Category-wise revenue

```
SQL> SELECT C.Category_Name,
  2         SUM(OD.Quantity * OD.Price) AS Revenue
  3  FROM Category C
  4  JOIN Product P ON C.Category_ID = P.Category_ID
  5  JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  6  GROUP BY C.Category_ID, C.Category_Name;

CATEGORY_NAME           REVENUE
-------------------- ----------
Celebration Cakes          3450
Morning Pastries            600
Artisan Breads             1350
```

Top-selling bakery items

```
SQL> SELECT *
  2  FROM
  3  (
  4      SELECT P.Title,
  5             SUM(OD.Quantity) AS Quantity_Sold
  6      FROM Product P
  7      JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  8      GROUP BY P.Product_ID, P.Title
  9      ORDER BY Quantity_Sold DESC
 10  )
 11  WHERE ROWNUM <= 5;

TITLE                     QUANTITY_SOLD
------------------------- -------------
Chocolate Fudge Cake                  3
Red Velvet Pastry                     3
Sourdough Bread                       2
Black Forest Cake                     2
```

Report 4: Business Decision Report

High-performing bakery items

```
SQL> SELECT P.Title,
  2         SUM(OD.Quantity) AS Quantity_Sold
  3  FROM Product P
  4  JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  5  GROUP BY P.Product_ID, P.Title
  6  HAVING SUM(OD.Quantity) >
  7  (
  8      SELECT AVG(Total_Qty)
  9      FROM
 10      (
 11          SELECT SUM(Quantity) AS Total_Qty
 12          FROM Order_Details
 13          GROUP BY Product_ID
 14      )
 15  );

TITLE                     QUANTITY_SOLD
------------------------- -------------
Chocolate Fudge Cake                  3
Red Velvet Pastry                     3
```

Low-performing bakery items

```
SQL> SELECT P.Title,
  2         NVL(SUM(OD.Quantity), 0) AS Quantity_Sold
  3  FROM Product P
  4  LEFT JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  5  GROUP BY P.Product_ID, P.Title
  6  ORDER BY Quantity_Sold ASC;

TITLE                     QUANTITY_SOLD
------------------------- -------------
Special Wedding Cake                  0
Croissants                            0
Black Forest Cake                     2
Sourdough Bread                       2
Red Velvet Pastry                     3
Chocolate Fudge Cake                  3

6 rows selected.
```

High-value customers

```
SQL> SELECT C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  2         SUM(O.Total_Amount) AS Total_Spending
  3  FROM Customer C
  4  JOIN Orders O ON C.Customer_ID = O.Customer_ID
  5  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  6  ORDER BY Total_Spending DESC;

CUSTOMER_NAME        TOTAL_SPENDING
-------------------- --------------
John Doe                       3590
Alice Brown                    1450
Bob White                      1250
Charlie Green                   950
Jane Smith                      720
David Miller                    500

6 rows selected.
```

Revenue trends

```
SQL> SELECT TO_CHAR(Order_Date, 'MON-YYYY') AS Month,
  2         SUM(Total_Amount) AS Revenue
  3  FROM Orders
  4  GROUP BY TO_CHAR(Order_Date, 'MON-YYYY')
  5  ORDER BY MIN(Order_Date);

MONTH                REVENUE
---------------- ----------
MAR-2026               7960
NOV-2026                500
```